

Reviewer Pack — Minimal Local Indefinite Integral and Resolution Proof Width...

Entry ab358a8d3f4d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 10:31:28 UTC

Minimal Local Indefinite Integral and Resolution Proof Width Correlation via Non-Commutative Geometry

Entry ID: ab358a8d3f4d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 10:31:28 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry and Algebra **Field B** (complexity object): Resolution Proof Complexity

Statement:

For any given CNF formula φ with n variables, the minimal local indefinite integral (LII) of its associated noncommutative space $N(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{LII}(N(\varphi)) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Noncommutative geometry has been shown to provide novel insights into the study of complex systems. By mapping CNF formulas onto noncommutative spaces, we can potentially uncover hidden structures related to proof complexity. The local indefinite integral (LII) in a noncommutative space captures information about the geometry of the space, which might be correlated with the complexity of resolving the formula.

Taxonomy category: Noncommutative_Geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a27a58652be50285

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For CNF formula φ with n variables, if Pearson correlation coefficient over 30 random seeds is ≥ 0.8 AND metric mean ≤ 3 , support $\text{LII}(N(\varphi)) = \Theta(w(\varphi))$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Minimal Local Indefinite Integral AND resolution proof complexity - arxiv:math-ph OR arxiv:math.AC AND noncommutative geometry - arxiv:quant-ph AND LII correlation with resolution proof width

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9809072v1] Complex Monopoles in the Path Integral - [http://arxiv.org/abs/1510.08876v2] Feynman integral in $\mathbb{R}^1 \oplus \mathbb{R}^m$ and complex expansion of ${}_2F_1$ - [http://arxiv.org/abs/quant-ph/0405018v2] Improved Bounds on the Randomized and Quantum Complexity of Initial-Value Problems - [http://arxiv.org/abs/2206.07611v1] Comment on “Nonperturbative Calculation of Born-Infeld Effects on the Schrodinger Spectrum of the Hydrogen Atom” [Phys. - [http://arxiv.org/abs/0804.3959v2] Nonholonomic systems on Lie algebroids - [http://arxiv.org/abs/0707.1781v1] Calculation of the metric in the Hilbert space of a PT-symmetric model via the spectral theorem - [http://arxiv.org/abs/quant-ph/0310042v2] Experimental evidence for bounds on quantum correlations - [http://arxiv.org/abs/1607.03106v1] Quantum Enhanced Correlation Matrix Memories via States Orthogonalisation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.7s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(x != -y for x, y in zip(clause, clause[1:])):
                clauses.append(clause)
        return clauses

    def compute_lii(cnf):
        n = len(cnf[0])
        lii = 0
        for clause in cnf:
            for literal in clause:
                if literal > 0:
                    lii += 1
        return lii / (n * len(cnf))
```

```

def compute_resolution_width(cnf):
    width = 0
    for clause in cnf:
        width = max(width, len(clause))
    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    lii = compute_lii(cnf)
    width = compute_resolution_width(cnf)
    results.append({
        "n": n,
        "lii": lii,
        "width": width
    })

mean_lii = sum(result["lii"] for result in results) / len(results)
mean_width = sum(result["width"] for result in results) / len(results)
correlation_coefficient = 0

if len(results) > 1:
    numerator = sum((result["lii"] - mean_lii) * (result["width"] - mean_width) for result in results)
    denominator = math.sqrt(sum((result["lii"] - mean_lii)**2 for result in results)) * math.sqrt(sum((result["width"] - mean_width)**2 for result in results))
    correlation_coefficient = numerator / denominator

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_lii <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"metric_value\": {trial_result['metric_value']}\"}}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    else:
        print(f"RESULT: NOT SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")

```

```

first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con]
print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8 or mean_lii > 3\"

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16332
2	preregistration	ollama_remote	glm4:latest	0	0	9015
3	novelty	ollama_remote	glm4:latest	0	0	8092
4	novelty	ollama_remote	glm4:latest	0	0	10122
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11844
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20318
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25134
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10722
9	judge	ollama_remote	glm4:latest	0	0	50872

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162451 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ab358a8d3f4d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ab358a8d3f4d.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ab358a8d3f4d.tar.gz (if generated)